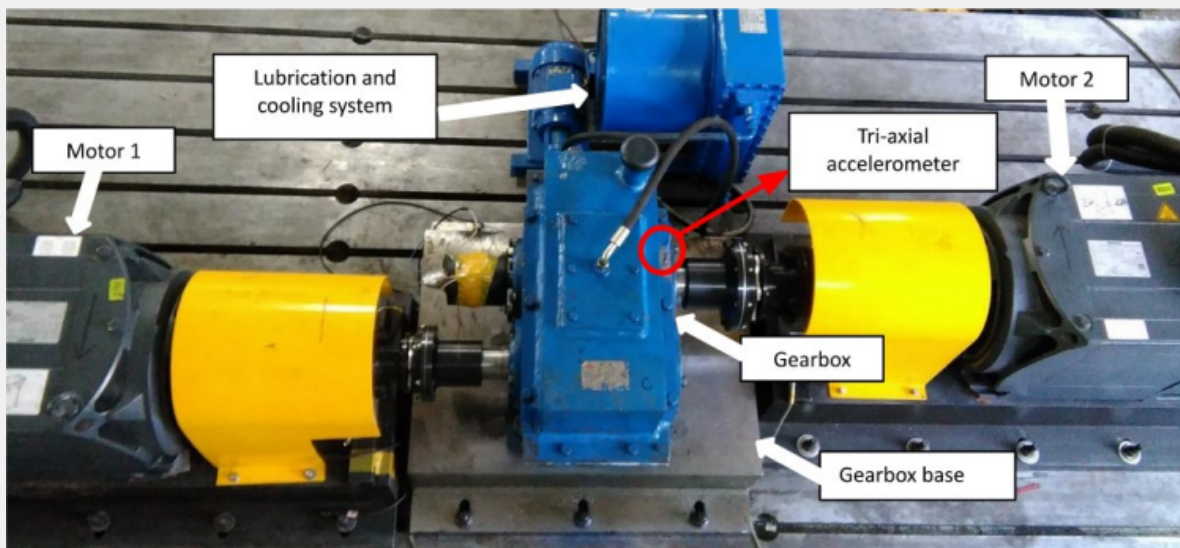


PHM AMERICA 2023

Manutenzione preventiva per la robotica e l'automazione intelligente



Autori:

Gabriel Piercecchi

Tosca Pierro

Anno accademico 2023-2024

Indice

1	Introduzione	1
1.1	Descrizione del Dataset	1
1.2	Condizioni Operative e Denominazione dei File	1
1.3	Caratteristiche del Segnale di Vibrazione	2
1.4	Durata dell'Acquisizione e Segmentazione	2
1.5	Dati di Degradazione e Livelli di Salute	3
1.6	Banco di Prova e Strumentazione	3
1.7	Obiettivi dell'Analisi	4
1.8	Procedure utilizzate	4
1.9	Repository	5
2	Addestramento CNN-AUTOENCODER	6
2.1	Introduzione	6
2.2	Procedimento	6
2.2.1	STEP 1 - Caricamento e Pre-elaborazione dei Dati	6
2.2.2	STEP 2 - Addestramento dell'Autoencoder CNN	6
2.2.3	STEP 3 - Caricamento dei dati unhealthy	8
2.2.4	STEP 4 - Valutazione degli Errori e Mapping in Livello	8
2.2.5	STEP 5 - Analisi dei Meta-dati Operativi	9
2.2.6	STEP 6 - Generazione dei File di Submission	11
2.2.7	STEP 7 - Caricamento dei dati di test e generazione del submission_test	11

2.2.8 STEP 8 - Tuning e Analisi dei Threshold	12
2.3 Osservazioni Finali	13
3 Addestramento CNN con SVM	15
3.1 Introduzione	15
3.2 Parametri e Configurazione	15
3.3 Caricamento e Segmentazione dei Dati	16
3.3.1 Preprocessing e Normalizzazione	16
3.3.2 Suddivisione Training/Validation e Definizione del Modello CNN	16
3.3.3 Visualizzazione e Valutazione del Modello	17
3.3.4 Predizione sul Test Set	17
3.4 Modulo di Anomaly Detection Esteso	17
3.4.1 Calcolo classe predetta	18
3.5 Esportazione dei Risultati	18
4 Risultati	19
4.1 Addestramento	19
4.2 Confusion matrix	20
4.3 Prestazioni del modello	21
4.3.1 Metriche di Classificazione per Classe	21
4.3.2 Distribuzione delle Probabilità Massime	21
4.3.3 Distribuzione delle occorrenze per colonna	22
4.4 Conclusioni	24

1. Introduzione

Lo studio del pitting negli ingranaggi rappresenta un aspetto molto importante nell'analisi dell'affidabilità e della manutenzione predittiva dei sistemi meccanici. In questo capitolo si introduce il dataset utilizzato per la diagnosi del pitting su un cambio monostadio, descrivendone la struttura, le condizioni operative, le caratteristiche dei segnali acquisiti e la metodologia di campionamento.

1.1 Descrizione del Dataset

Il dataset è composto da numerosi file di testo (.txt) raccolti durante esperimenti di pitting su un cambio monostadio. Ogni file contiene informazioni sulle vibrazioni e sul segnale tachometrico, ed è organizzato in quattro colonne:

- **Colonna 1:** Accelerazione orizzontale.
- **Colonna 2:** Accelerazione assiale.
- **Colonna 3:** Accelerazione verticale.
- **Colonna 4:** Segnale del tachimetro.

I segnali sono acquisiti in modo sincrono per garantire una corretta analisi delle vibrazioni e delle anomalie associate al degrado degli ingranaggi.

1.2 Condizioni Operative e Denominazione dei File

I file sono denominati secondo le condizioni operative, indicando la velocità di rotazione in giri/min e la coppia applicata in Nm. Ad esempio, un file denominato V100_200N_2.txt rappresenta dati acquisiti con 100 giri/min in ingresso e 200 Nm di coppia sull'albero di uscita, dove il suffisso “_2” indica la seconda ripetizione dell'esperimento.

1.3 Caratteristiche del Segnale di Vibrazione

I segnali di vibrazione sono acquisiti con una frequenza di campionamento di $20480Hz$, valore scelto per catturare in dettaglio le componenti dinamiche rilevanti, rispettando il teorema di Nyquist. Il tachimetro, montato sull'albero di uscita, fornisce un impulso per ogni rotazione, consentendo la segmentazione precisa del segnale.

1.4 Durata dell'Acquisizione e Segmentazione

La durata dell'acquisizione varia in funzione della velocità di rotazione:

- **100-200 rpm:** circa 12 secondi di dati.
- **300-1000 rpm:** circa 6 secondi.
- **1200 rpm e oltre:** circa 3 secondi.

NOTA Per garantire un bilanciamento equo dei dati di addestramento ed evitare squilibri nei pesi dei campioni, è stata sviluppata la funzione `processFilesForSegmentation` (file `processFilesForSegmentation.m`), la quale ha elaborato e memorizzato segmenti uniformi di 3 secondi per ciascun campione.

Nel caso si volesse riprodurre il codice costruito, è necessario commentare la parte di codice che richiama la suddetta funzione dopo la prima esecuzione. Di seguito il codice:

```
...
%% Preparazione dei dati
% Per processare i file di training:
% !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
% ATTENZIONE: I file di training devono essere processati una sola volta.
% COMMENTARLI DOPO LA PRIMA ESECUZIONE DI QUESTO SCRIPT O DI complete_cnn_sum.m
% !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

processFilesForSegmentation('B - PHM America 2023 -
Dataset\Data_Challenge_PHM2023_training_data\Pitting_degradation_level_0 (Healthy)\')
;
processFilesForSegmentation('B - PHM America 2023 -
```

```
Dataset\Data_Challenge_PHM2023_training_data\Pitting_degradation_level_1\');  
processFilesForSegmentation('B - PHM America 2023 -  
Dataset\Data_Challenge_PHM2023_training_data\Pitting_degradation_level_2\');  
processFilesForSegmentation('B - PHM America 2023 -  
Dataset\Data_Challenge_PHM2023_training_data\Pitting_degradation_level_3\');  
processFilesForSegmentation('B - PHM America 2023 -  
Dataset\Data_Challenge_PHM2023_training_data\Pitting_degradation_level_4\');  
processFilesForSegmentation('B - PHM America 2023 -  
Dataset\Data_Challenge_PHM2023_training_data\Pitting_degradation_level_6\');  
processFilesForSegmentation('B - PHM America 2023 -  
Dataset\Data_Challenge_PHM2023_training_data\Pitting_degradation_level_8\');  
  
% Per processare i file di test:  
processFilesForSegmentation('B - PHM America 2023 -  
Dataset\Data_Challenge_PHM2023_test_data\');  
  
% Per processare i file di validation:  
processFilesForSegmentation('B - PHM America 2023 -  
Dataset\Data_Challenge_PHM2023_validation_data\');  
...
```

1.5 Dati di Degradazione e Livelli di Salute

Gli ingranaggi sono stati sottoposti a pitting controllato, con livelli di degradazione variabili da 0 (sano) a 10 (massima degradazione). Il dataset di training comprende 78 condizioni operative distribuite su 7 livelli di salute, con alcune esclusioni (pitting 5, 7, 9, 10) per l'addestramento. Ogni condizione è registrata con 5 ripetizioni, per un totale di 2016 file.

1.6 Banco di Prova e Strumentazione

L'esperimento è condotto su un cambio monostadio con ingranaggi cilindrici:

- **Ingranaggio di comando:** 40 denti.
- **Ingranaggio condotto:** 72 denti.

Il banco di prova utilizza due servomotori *Siemens* da $45kW$ (uno per l'azionamento e uno per il carico) e un accelerometro triassiale fissato sulla scatola del cambio vicino al cuscinetto di uscita.

1.7 Obiettivi dell'Analisi

L'obiettivo del problema è sviluppare un modello in grado di stimare la gravità del guasto a partire da dati raccolti in condizioni operative eterogenee. In particolare, il modello deve generalizzare su livelli di guasto e condizioni operative non presenti nel set di addestramento, poiché i dati di training coprono solo uno stato sano e sei livelli di guasto, mentre i set di test e validazione comprendono undici livelli di salute. Inoltre, il sistema deve fornire una misura di fiducia nelle sue previsioni, in modo da poter valutare l'attendibilità dei risultati prima di procedere con azioni successive.

Nei capitoli successivi verranno approfondite le tecniche di analisi e i risultati ottenuti.

1.8 Procedure utilizzate

Nel presente lavoro sono stati sviluppati due approcci distinti per la rilevazione di anomalie e la stima del fault level nei segnali vibratorii.

Il primo approccio si basa sull'uso di un **autoencoder** CNN addestrato in modalità unsupervised sui dati healthy. In questo metodo, il modello impara a ricostruire fedelmente i segnali normali e, durante il testing, l'errore di ricostruzione viene utilizzato per individuare anomalie. Se l'errore supera una soglia (definita come la media degli errori healthy più tre volte la loro deviazione standard), il segmento viene considerato anomalo e mappato in un fault level proporzionale alla deviazione. Inoltre, per ogni segmento viene generata una distribuzione di probabilità (tramite una funzione gaussiana) e viene assegnata una confidenza binaria.

Il secondo approccio, invece, adotta una strategia diversa: una CNN viene addestrata in modalità supervisionata per classificare i segnali (trattati come immagini) e, successivamente, si integra un modulo di anomaly detection basato su **one-class SVM**. In questo caso, la CNN estrae feature rilevanti dai segnali e, tramite il layer intermedio, queste vengono utilizzate per addestrare i modelli one-class SVM sulle classi "base" (ad esempio "4", "6" e "8"). Durante il testing, se un segmento clas-

sificato in una classe base presenta un punteggio di anomalia inferiore a una soglia prestabilita, il sistema interpreta il campione come appartenente a una classe mancante (ad esempio, riassegnandolo da “4” a “5”).

NOTA In entrambi gli approcci, i file utilizzati sono stati scaricati dai seguenti link: https://phm-datasets.s3.amazonaws.com/Data_Challenge_PHM2023_training_data.zip per i dati di training, https://phm-datasets.s3.amazonaws.com/Data_Challenge_PHM2023_test_data.zip per i dati di testing e https://phm-datasets.s3.amazonaws.com/Data_Challenge_PHM2023_validation_data.zip per i dati di validation. Una volta scaricati, sono stati estratti e salvati all’interno della cartella B - PHM America 2023 - Dataset, nello stesso percorso dei file MATLAB.

Nel caso si volesse riprodurre il seguente progetto, è imperativo seguire la stessa procedura.

1.9 Repository

LA repository del seguente progetto è presente al seguente link: <https://github.com/GabrielPiercecchi/ProgettoManutenzionePreventivaR-AI>. Per scaricare il codice è sufficiente cliccare sul button <> Code e successivamente su "Download ZIP".

2. Addestramento CNN-AUTOENCODER

2.1 Introduzione

Il modello che andremo a descrivere in questo capitolo, rappresenta una pipeline completa per l'anomaly detection e la stima del fault level, espresso su una scala da 0 a 10. L'approccio sfrutta un **autoencoder CNN** in modalità unsupervised, combinato con una semplice estrazione delle feature. L'obiettivo principale è sviluppare un modello in grado di apprendere il comportamento "sano" (healthy) e, successivamente, riconoscere e quantificare le anomalie nei dati raccolti in condizioni operative eterogenee.

NOTA Il nome del file relativo a questo modello è `complete_autoencoder_CNN_pipeline.m`

2.2 Procedimento

2.2.1 STEP 1 - Caricamento e Pre-elaborazione dei Dati

Il processo inizia con il caricamento dei dati "healthy", ossia quelli relativi al livello 0 di pitting, prelevati da una cartella specifica. Una funzione filtra e carica i file in formato `.txt`, registrando un totale di 287 file healthy. Su questi dati viene applicata una dummy feature extraction che non altera il contenuto originale, seguita dalla segmentazione dei segnali in finestre di 3 secondi. La funzione per la segmentazione suddivide ciascun file in segmenti uniformi, ottenendo esattamente 287 segmenti, così da garantire la compatibilità con l'autoencoder CNN.

2.2.2 STEP 2 - Addestramento dell'Autoencoder CNN

A seguito della pre-elaborazione, viene definita l'architettura dell'autoencoder CNN. Il layer di input è definito con dimensioni pari a `[samplesPerSeg, numChannels, 1]`, dove `samplesPerSeg` indica il numero di campioni per segmento (20480) e `numChannels` il numero di canali (ad esempio, `acc_x`, `acc_y`, `acc_z` e tachimetro).

L'encoder, costituito da strati convoluzionali, ReLU e pooling, riduce la dimensionalità del segnale, mentre il decoder utilizza strati di convoluzione trasposta per ricostruire l'input.

Il modello viene addestrato in modalità unsupervised utilizzando i dati healthy come riferimento; durante il training, MATLAB mostra la finestra di "training progress" (Figura 2.1).

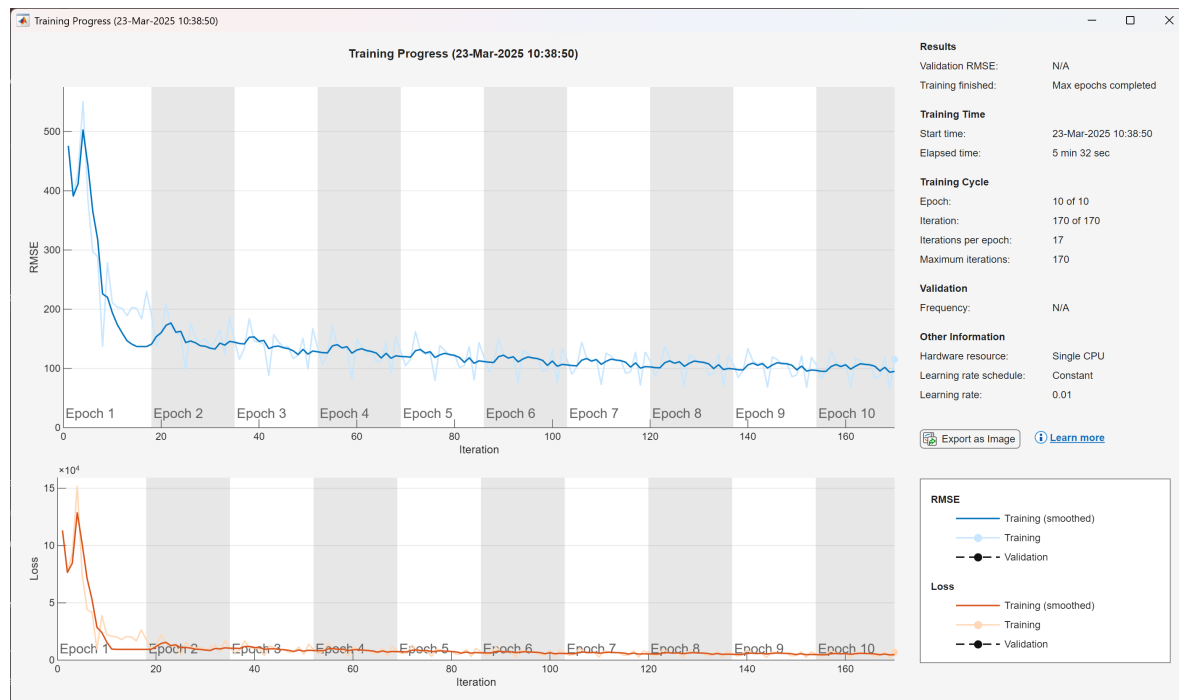


Figura 2.1: Training Progress

Il training è stato condotto su singola CPU e, dopo circa 10 epoche, il modello ha completato l'addestramento con una soglia di errore healthy pari a 0.4573.

Dopo l'addestramento, autoencoder viene utilizzato per ricostruire i dati healthy. L'errore di ricostruzione viene quindi calcolato come la media degli errori quadrati (MSE) tra ogni segmento di input e il corrispondente output ricostruito. Successivamente, viene generato un istogramma per visualizzare la distribuzione dell'errore di ricostruzione healthy (Figura 2.2).

Infine, la soglia di errore, utile per distinguere in seguito i dati anomali, viene impostata come la media dell'errore di ricostruzione più tre volte la deviazione standard.

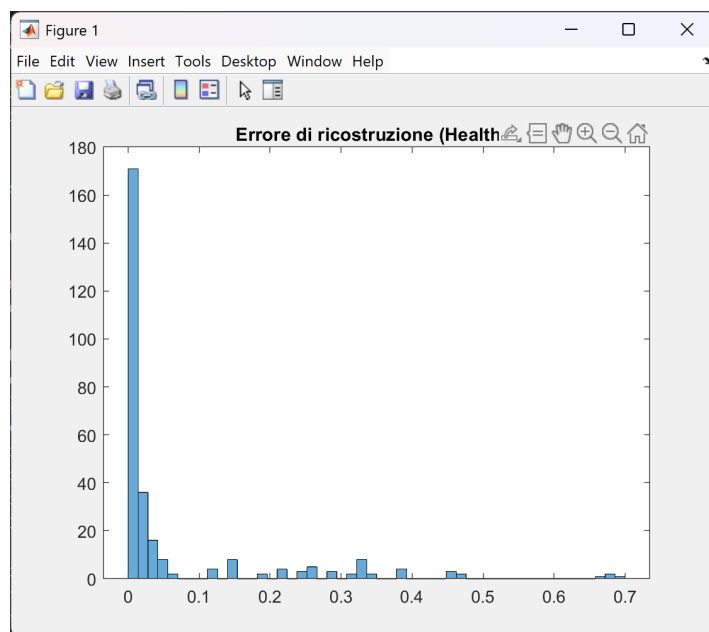


Figura 2.2: Istogramma dell'errore di ricostruzione sui dati healthy

2.2.3 STEP 3 - Caricamento dei dati unhealthy

Terminato l'addestramento, il modello viene applicato ai dati "unhealthy" (cioè a quelli con livelli di degrado 1, 2, 3, 4, 6 e 8). In questa fase le feature vengono estratte dai dati unhealthy. Analogamente alla fase relativa ai dati healthy, in questo primo tentativo l'operazione di estrazione è "dummy", nel senso che i dati vengono processati senza trasformazioni complesse, ma vengono semplicemente passati attraverso la funzione.

Il passo successivo consiste nel segmentare i dati unhealthy in finestre temporali, in modo da preparare gli input per eventuali elaborazioni successive.

2.2.4 STEP 4 - Valutazione degli Errori e Mapping in Livello

L'autoencoder, già addestrato sui dati healthy, calcola l'errore di ricostruzione per ogni segmento, misurato come la differenza quadratica media tra input e output. Se l'errore supera 0.4573, il segmento viene considerato anomalo e mappato in un fault level proporzionale alla deviazione; in questo caso il livello medio predetto risulta pari a circa 0.01, con il 100% dei segmenti aventi alta confidenza.

Nello specifico, il mapping in un livello di degradazione si basa sul confronto dell'errore di ricostruzione calcolato con una soglia definita come la media degli

errori healthy a cui si aggiungono tre volte la deviazione standard. Se l'errore di un segmento supera tale soglia, il segmento viene considerato anomalo e gli viene assegnato un fault level proporzionale alla differenza rispetto alla soglia. Per ciascun segmento, viene inoltre generata una distribuzione di probabilità tramite una funzione gaussiana centrata sul livello predetto, e viene assegnata una confidenza binaria. Tale procedura è illustrata in Figura 2.3, dove l'istogramma dell'errore di ricostruzione sui dati unhealthy evidenzia chiaramente la linea della soglia.

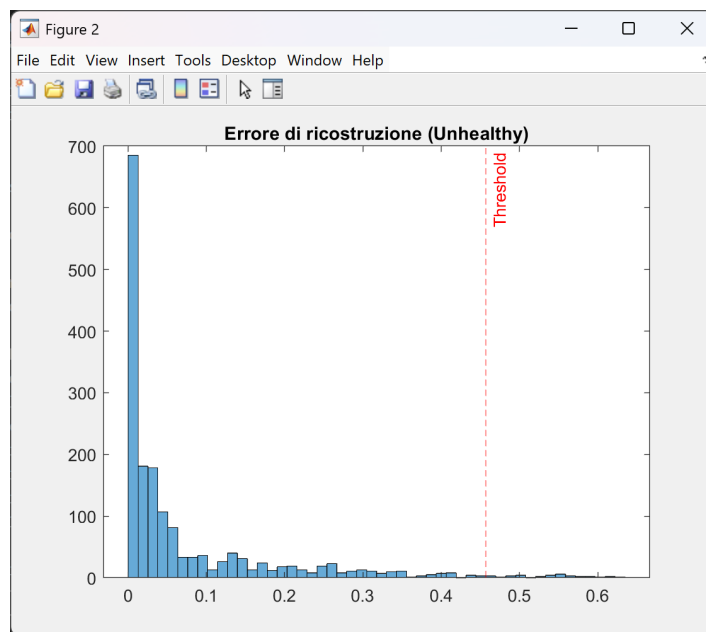


Figura 2.3: Istogramma dell'errore di ricostruzione sui dati unhealthy, con la linea della soglia evidenziata

2.2.5 STEP 5 - Analisi dei Meta-dati Operativi

Si procede analizzando i meta-dati operativi, come la velocità del motore e il torque, per valutare la correlazione con l'errore medio di ricostruzione a livello di file. I grafici a dispersione mostrano chiaramente:

- In 800 file test (segmentati in 800 segmenti) il mapping e la predizione con l'autoencoder risultano analoghi a quelli dei dati unhealthy, con livelli medi predetti pari a 0.01 e confidenza elevata (100%).
- Le relazioni tra Motor Speed, Torque ed errore medio sono illustrate nelle Figure 2.4, 2.5 e 2.6, fornendo una visione integrata delle condizioni operative.

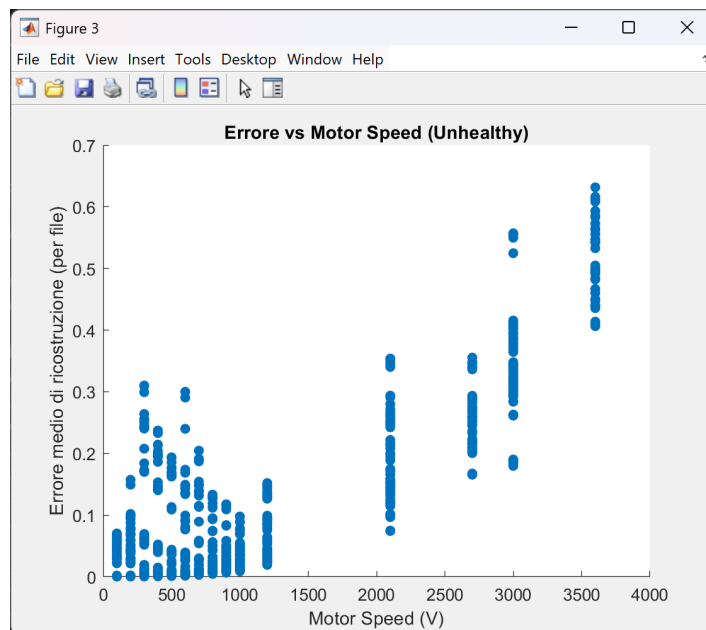


Figura 2.4: Scatter plot “Errore vs Motor Speed (Unhealthy)”, che mostra la relazione tra velocità del motore e errore medio di ricostruzione

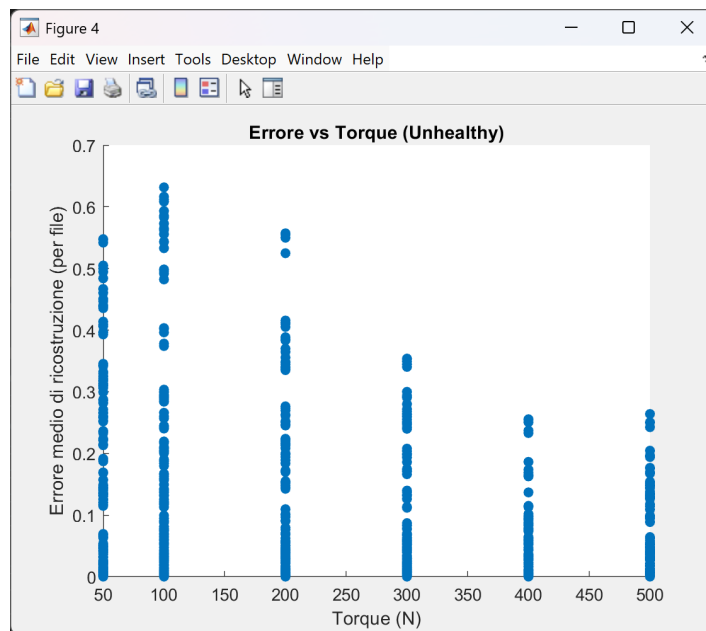


Figura 2.5: Scatter plot “Errore vs Torque (Unhealthy)”, per evidenziare la correlazione tra torque e errore di ricostruzione

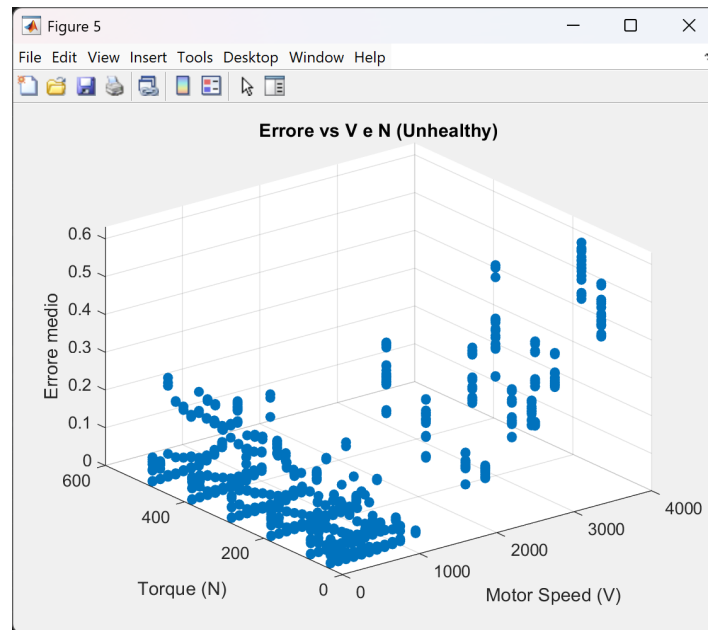


Figura 2.6: Scatter plot 3D “Errore vs V e N (Unhealthy)”, che offre una visione integrata di come velocità e torque interagiscano nell’influencare l’errore medio

2.2.6 STEP 6 - Generazione dei File di Submission

Nel sesto step della pipeline il sistema raccoglie e organizza le informazioni ottenute dal modello (identificativo del segmento, distribuzione di probabilità per i livelli di degrado e confidenza della previsione) in una tabella, che viene poi esportata in un file CSV (*submission_cnn.csv*). Tale file di submission rappresenta il prodotto finale di questa parte della pipeline, pronto per essere utilizzato nelle fasi successive della valutazione del modello o per l’analisi comparativa dei risultati.

2.2.7 STEP 7 - Caricamento dei dati di test e generazione del submission_test

Il modello viene poi applicato ai dataset di test. In fase di test, sono stati caricati 800 file e, dopo segmentazione e predizione, il fault level medio risulta nuovamente pari a 0.01 con alta confidenza (100%). Una volta ottenuti i segmenti di test, il modello addestrato viene utilizzato per predire la ricostruzione di ciascun segmento. La differenza tra il segnale originale e quello ricostruito viene misurata calcolando il *Mean Squared Error (MSE)* per ogni segmento, ottenendo così un vettore di errori di ricostruzione. La distribuzione degli errori viene poi visualizzata mediante

un istogramma (Figura 2.7), arricchito da una linea rossa che rappresenta la soglia definita in precedenza, per evidenziare quali segmenti superino il limite di errore ritenuto accettabile.



Figura 2.7: Istogramma dell'errore di ricostruzione sui dati di test

I risultati ottenuti vengono organizzati in una matrice che include, per ogni segmento, un identificativo univoco, la distribuzione di probabilità sui livelli di degrado e il valore di confidenza. Questa matrice viene convertita in una tabella MATLAB, a cui vengono assegnati dei nomi di colonna esplicativi (ad esempio, 'sample_id', 'prob_0', ..., 'prob_10', 'confidence'). Il passo conclusivo prevede l'esportazione della tabella in un file CSV, denominato "submission_cnn.csv", che costituisce il prodotto finale della fase di test.

2.2.8 STEP 8 - Tuning e Analisi dei Threshold

In questa sezione vengono eseguite operazioni di tuning e analisi, con l'obiettivo di esplorare soglie alternative per l'errore di ricostruzione sui dati healthy e di raccogliere statistiche chiave. Questo processo aiuta a comprendere meglio la distribuzione degli errori e a valutare come differenti criteri di soglia possano influenzare l'identificazione delle anomalie.

L'analisi riporta che:

- La media degli errori healthy è pari a 0.0665, con una deviazione standard di 0.1303. Tuttavia, la soglia scelta per identificare anomalie ($media + 3 \times std = 0.4573$) risulta relativamente alta rispetto alla media, il che potrebbe portare il modello a considerare "normali" anche alcuni segmenti con un errore elevato, riducendo la sensibilità nella rilevazione dei guasti lievi.
- Sono state esplorate soglie alternative, rispettivamente $media + 2 \times std = 0.3270$ e $media + 4 \times std = 0.5876$, per valutare il bilanciamento tra falsi positivi e falsi negativi.

Queste soglie vengono sovrapposte sull'istogramma degli errori healthy (Figura 2.8), evidenziando il bilanciamento tra falsi positivi e falsi negativi.

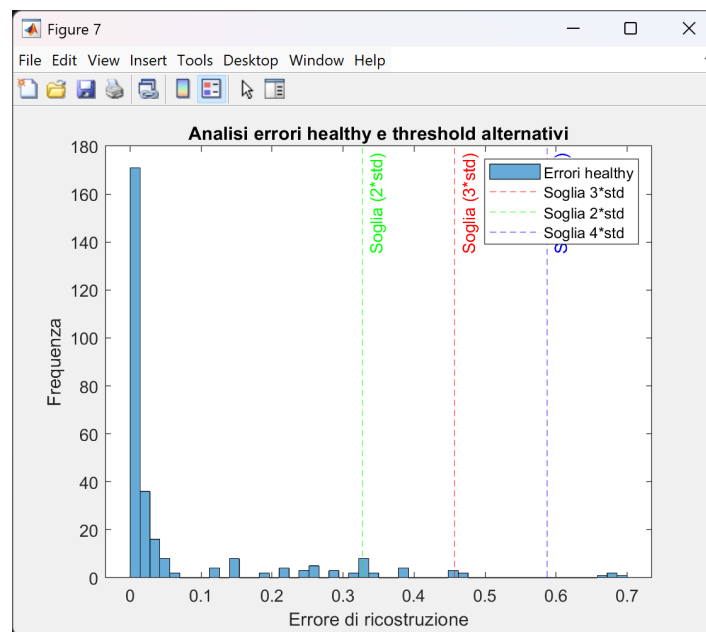


Figura 2.8: Istogramma degli errori healthy con le soglie alternative ($2 \times std$, $3 \times std$ e $4 \times std$) evidenziate

Alla fine, le feature estratte dai dati healthy e unhealthy vengono combinate e salvate in un file MAT.

2.3 Osservazioni Finali

Nonostante l'analisi delle soglie, il metodo presenta alcune limitazioni. Una delle principali criticità riguarda la **scarsa generalizzazione sui dati unhealthy**: il modello è stato addestrato esclusivamente su dati healthy, il che potrebbe impedirgli di

catturare correttamente la distribuzione degli errori nei dati guasti. Di conseguenza, la soglia scelta potrebbe non separare in modo efficace le condizioni normali da quelle anomale, portando a errori di classificazione.

Infine, un'ulteriore limitazione è la **mancanza di validazione** con dati etichettati unhealthy. Non essendo stata effettuata una verifica diretta su dati con etichette di guasto, non è possibile determinare con precisione il tasso di falsi positivi e falsi negativi. Questo rende difficile valutare l'effettiva efficacia del modello nel distinguere tra healthy e unhealthy, suggerendo la necessità di un approccio più robusto per migliorare l'accuratezza delle previsioni.

Questo ha comportato l'abbandono di questo modello in favore di uno nuovo che fosse più preciso nelle previsioni.

3. Addestramento CNN con SVM

3.1 Introduzione

In questo capitolo verrà analizzato un metodo alternativo per la stima della gravità dei guasti e il rilevamento delle anomalie. In questo approccio, ogni segmento di dati viene trattato come un'immagine con dimensioni $[20000 \times 3 \times 1]$.

I file di input, forniti in formato testo, contengono quattro colonne, tuttavia vengono utilizzate solo le prime tre, che corrispondono alle coordinate spaziali X , Y e Z .

Il modulo implementa un sistema di *anomaly detection*, il quale interviene in particolare sulla classe "2" e sulle classi di base (4, 6 e 8). In presenza di anomalie, queste ultime vengono riassegnate alle classi mancanti (5, 7 e 9), garantendo così una migliore rappresentazione delle condizioni di guasto nel dataset.

NOTA Il nome del file relativo a questo modello è `complete_cnn_svm.m`

3.2 Parametri e Configurazione

Inizialmente vengono definiti i parametri fondamentali per l'elaborazione e l'addestramento di un modello di classificazione. La frequenza di campionamento è impostata a 20480 Hz, in modo che ogni secondo di segnale contenga 20480 campioni, dato che la durata di ciascun segmento è fissata a 1 secondo. Il numero di campioni per segmento viene calcolato come il prodotto tra la frequenza di campionamento e la durata del segmento.

Il problema di classificazione viene formulato su 7 classi, ottenute rietichettando le classi originali (0, 1, 2, 3, 4, 6, 8) in modo che risultino consecutive. Sono inoltre definiti il batch size (32) e il numero di epoche (15) per il processo di ottimizzazione. Infine, vengono specificati i percorsi per i dati di training e di test e viene creata una mappa (`containers.Map`) per associare i nomi delle cartelle alle rispettive etichette numeriche.

3.3 Caricamento e Segmentazione dei Dati

Per la segmentazione, una funzione riceve in ingresso il percorso della cartella contenente i file di test e la lunghezza di ciascun segmento (in campioni). Il codice individua tutti i file di testo presenti nella directory e, per ognuno di essi, tenta di caricare i dati. Se un file contiene almeno tre colonne, vengono selezionate le prime tre; altrimenti, viene generato un errore che interrompe l'elaborazione del file.

Per ogni file, viene calcolato il numero di segmenti ottenibili dividendo il numero totale di campioni per la lunghezza del segmento, escludendo quelli incompleti. Ogni segmento, estratto come intervallo di dati, viene concatenato lungo la terza dimensione di un array e, parallelamente, viene memorizzato il nome del file di origine in un cell array.

Al termine, la funzione restituisce un array tridimensionale organizzato come `[segmentLength x 3 x numSegments]` insieme alle informazioni sui file.

3.3.1 Preprocessing e Normalizzazione

I dati di training vengono normalizzati canale per canale: per ciascuna delle tre colonne vengono calcolate la media e la deviazione standard, valori poi utilizzati per standardizzare sia i dati di training che quelli di test. Successivamente, i dati, originariamente organizzati in array 3D, vengono convertiti in formato 4D (`[segmentLength, 3, 1, N]`) per garantire la compatibilità con la rete neurale convoluzionale. Le etichette di training sono, infine, convertite in variabili categoriche per facilitare il processo di apprendimento supervisionato.

3.3.2 Suddivisione Training/Validation e Definizione del Modello CNN

Il dataset di training viene suddiviso in un set per l'addestramento (80%) e uno per la validazione (20%) mediante la funzione `cvpartition`. Il modello CNN viene definito attraverso una serie di layer:

- Un `imageInputLayer` per l'input, seguito da strati di convoluzione, attivazione (ReLU) per introdurre non linearità e pooling che riducono la dimensionalità spaziale dei dati, facilitando l'estrazione delle caratteristiche più rilevanti;
- Layer completamente connessi e un `dropoutLayer` per ridurre l'overfitting;

- Un layer softmax e un `classificationLayer` per la classificazione finale.

L'addestramento della rete avviene in modalità supervisionata, utilizzando l'ottimizzatore `RMSprop`, un tasso di apprendimento iniziale predefinito, un numero specifico di epoche e una dimensione dei mini-batch fissata, con il monitoraggio delle prestazioni sul validation set.

3.3.3 Visualizzazione e Valutazione del Modello

Durante l'addestramento, le feature interne vengono estratte dal layer `'relu_fc1'` utilizzando la funzione `activations`. Successivamente, il modello viene valutato sul validation set tramite la funzione `classify`, si calcola l'accuratezza e si genera una matrice di confusione (tramite `confusionchart`), che viene salvata per ulteriori analisi, mostrata in Figura 4.2.

3.3.4 Predizione sul Test Set

Il modello viene utilizzato per predire le etichette dei segmenti del test set. Con `classify` si ottengono le etichette predette, mentre con `predict` si calcolano le probabilità associate a ciascuna classe.

Per ogni segmento, viene individuata la classe con la massima probabilità (confidenza) e vengono stampati a video il numero del segmento, il nome del file da cui proviene e il livello di guasto predetto con il relativo valore di confidenza. Questo fornisce un quadro dettagliato e interpretativo delle performance del modello sui dati di test.

3.4 Modulo di Anomaly Detection Esteso

Per migliorare la robustezza del sistema, è stato implementato un modulo di **anomaly detection esteso** che integra un ulteriore controllo sulle cosiddette classi "base". In particolare, il codice si concentra sulle classi "4", "6" e "8", che rappresentano i livelli di riferimento per cui, in caso di rilevamento di comportamenti anomali, il segmento viene riassegnato rispettivamente alle classi "5", "7" e "9" (livelli non presenti nel training).

Il procedimento inizia definendo le classi di base e creando una mappa (`containers.Map`) che associa ciascuna di esse alla corrispondente classe mancante. Successivamente, vengono estratte le feature dai dati di training dal layer `'relu_fc1'` utilizzando

la funzione `activations`. Queste rappresentazioni interne vengono poi impiegate per addestrare, tramite la funzione `fitcsvm` con kernel gaussiano, dei modelli **One-Class Support Vector Machine** (SVM). I modelli sono configurati per standardizzare i dati e rilevare una frazione di outlier pari al 5%, in modo da apprendere il comportamento "normale" per ciascuna classe base. Durante la fase di test, le feature vengono estratte nello stesso modo e, per ogni segmento inizialmente classificato come appartenente a una classe base, il relativo modello SVM calcola uno score. Se tale score è inferiore a una soglia predefinita (in questo caso 0.7), il segmento viene considerato anomalo e la sua etichetta viene riassegnata alla classe mancante corrispondente, secondo la mappa definita.

3.4.1 Calcolo classe predetta

In generale, se la classe predetta non è una classe mancante (5, 7 o 9), si individua il suo indice tra le classi viste e si assegna una probabilità pari a 1 nella colonna corrispondente della matrice. Se invece appartiene alle classi mancanti, si identifica la classe base corrispondente (4 per 5, 6 per 7, 8 per 9), si estrae la sua probabilità e si ricalcola la distribuzione, trasferendo parte della probabilità alla classe mancante e ridistribuendo il resto tra le altre classi. Se la classe base non è stata vista, si assegna direttamente una probabilità pari a 1 alla classe mancante come fallback. Questo approccio garantisce una distribuzione coerente delle probabilità e migliora la gestione delle classi assenti nel modello.

3.5 Esportazione dei Risultati

Al termine del processo di classificazione e rilevamento delle anomalie, i risultati del test vengono esportati in un file CSV (`submission_cnn_svm.csv`). Questo file contiene il nome del file (o l'ID del segmento), la predizione finale e il livello di confidenza associato alla predizione. È inoltre prevista un'esportazione opzionale dei risultati del validation set in un file separato (`validation_submission_cnn_svm.csv`), utile per ulteriori analisi e confronti. In questo modo, il sistema fornisce un output strutturato che consente di valutare le prestazioni complessive del modello e di identificare eventuali criticità nella classificazione.

4. Risultati

4.1 Addestramento

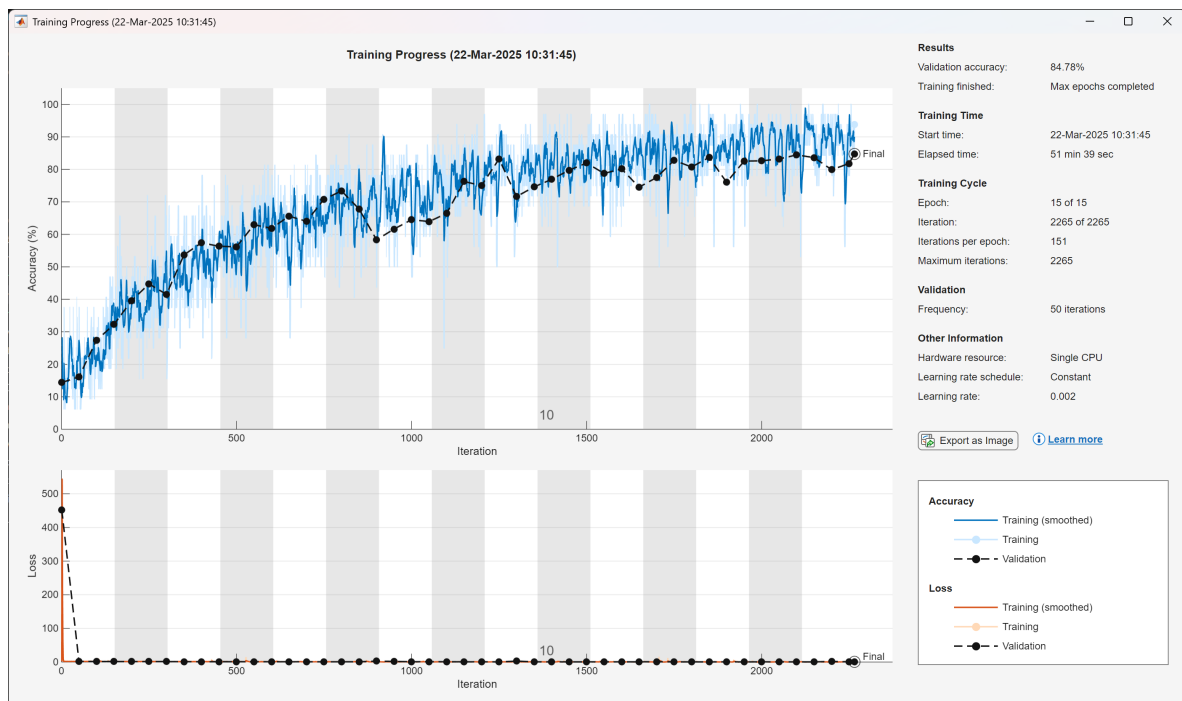


Figura 4.1: Accuracy e loss

Nel grafico mostrato nella Figura 4.1 si osserva l'andamento dell'accuratezza di training (in blu, con eventuale intervallo di oscillazione) e di validazione (in nero) in funzione delle iterazioni di addestramento. Fin dalle prime epoche, entrambe le curve mostrano una tendenza crescente, a indicare che la rete neurale convoluzionale sta gradualmente migliorando la propria capacità di riconoscere i pattern nei dati. Con il progredire delle iterazioni, l'accuratezza di training si mantiene leggermente superiore rispetto a quella di validazione, la differenza non appare eccessiva poiché l'accuratezza di validazione segue un trend analogo e si stabilizza su valori prossimi a quelli di training.

Nella parte inferiore del grafico, la funzione di costo (loss) decresce progressivamente, confermando la riduzione dell'errore complessivo man mano che la rete affina i propri parametri. L'andamento regolare mostra che il processo di ottimizzazione procede in modo efficace. Nella sezione “*Results*” del pannello sono riportati alcuni dati sintetici: tra questi, il valore finale di accuratezza di validazione dell'84,78%, l'epoca in cui è stato raggiunto il miglior risultato e il tempo totale di addestramento.

4.2 Confusion matrix

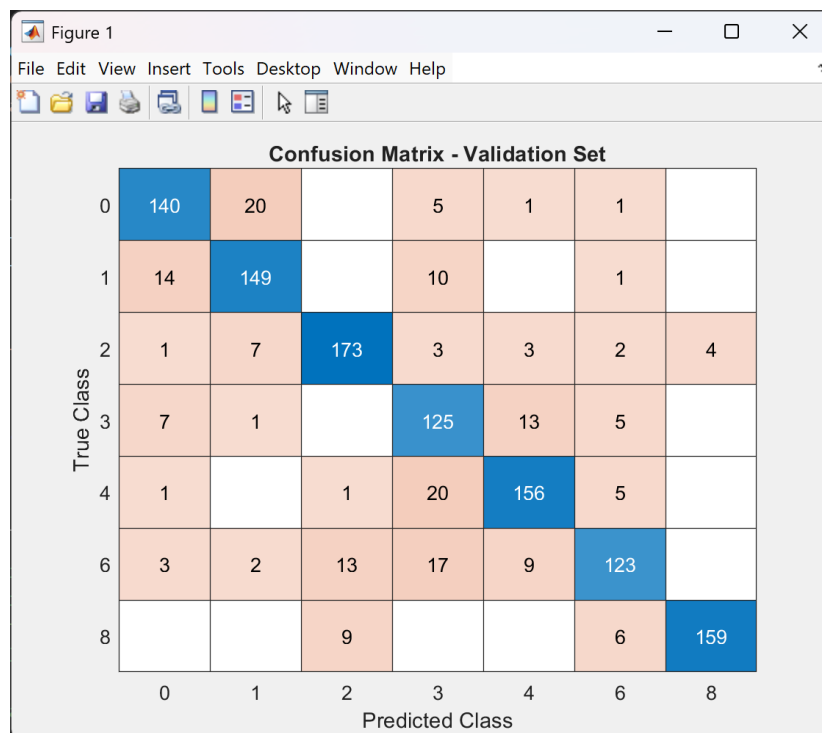


Figura 4.2: Confusion matrix

Dall'analisi della matrice di confusione in Figura 4.2 si osserva che i valori diagonali, nettamente superiori rispetto a quelli fuori diagonale, indicano una classificazione corretta della maggior parte dei segmenti per ciascuna classe. Nonostante alcune lievi confusioni residue, i risultati complessivi confermano che il modello appreso durante l'addestramento è in grado di distinguere in modo affidabile i vari livelli di gravità.

4.3 Prestazioni del modello

Per valutare le prestazioni del modello di classificazione, sono state analizzate due principali rappresentazioni grafiche:

4.3.1 Metriche di Classificazione per Classe

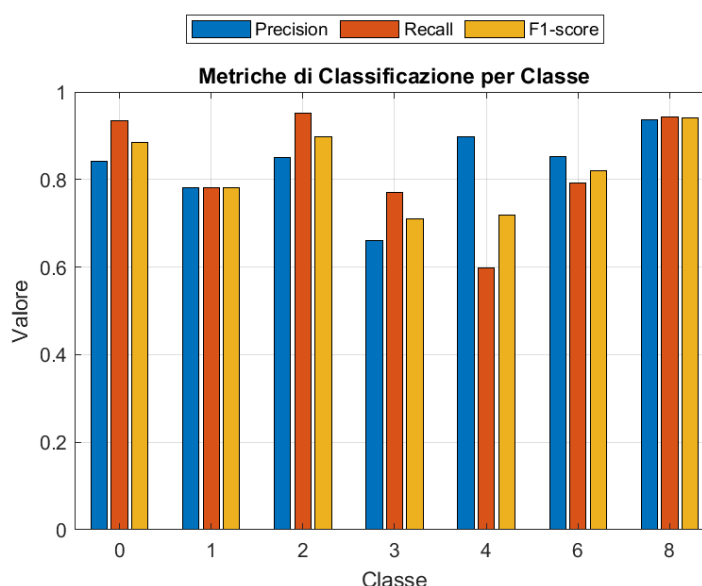


Figura 4.3: Metriche di Classificazione per Classe

Il grafico (Figura 4.3) rappresenta le metriche di *precision*, *recall* e *F1-score* per ciascuna classe. La precisione misura la frazione di predizioni corrette rispetto a tutte le predizioni effettuate per una determinata classe, il recall indica la capacità del modello di identificare correttamente gli esempi positivi, mentre l'*F1-score* bilancia questi due aspetti con una media armonica. Dai risultati emerge una buona performance generale del modello, con variazioni nelle diverse classi.

4.3.2 Distribuzione delle Probabilità Massime

Il grafico (Figura 4.4) mostra la distribuzione delle probabilità massime associate alle predizioni sia per il set di test che per il set di validazione. Quest'analisi valuta il livello di confidenza del modello nelle sue previsioni. L'istogramma e la stima

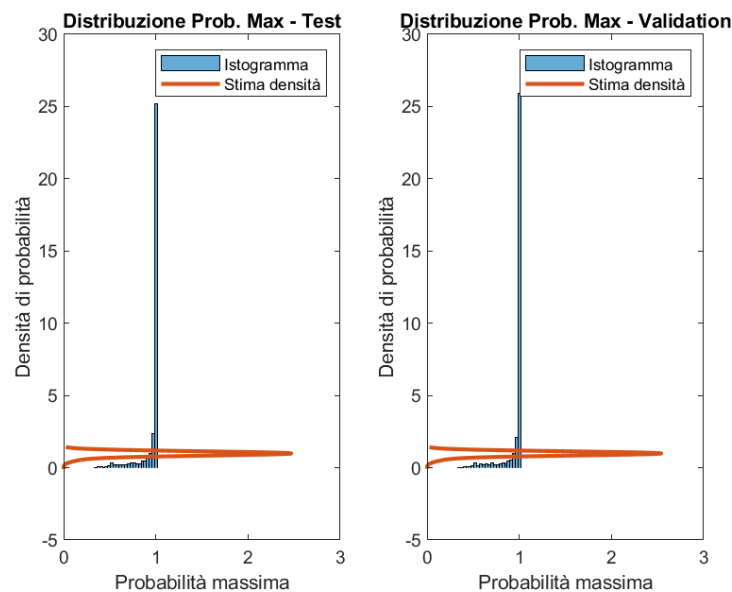


Figura 4.4: Distribuzione probabilità

della densità evidenziano un picco molto concentrato, suggerendo che il modello tende a effettuare previsioni con alta sicurezza.

4.3.3 Distribuzione delle occorrenze per colonna

Analizzando i dati forniti dalle tabelle mostrate in Figura 4.5 per i dati di test e in Figura 4.6 per i dati di validazione ottenuti dai corrispettivi file CSV, si può osservare quanto segue:

- Le colonne $prob_2$, $prob_7$ e $prob_9$ hanno il maggior numero di occorrenze come valore massimo, indicando che i livelli di guasto associati a queste probabilità sono predominanti nei dati.
- Le colonne $prob_5$, $prob_6$ e $prob_8$ presentano un numero molto basso di occorrenze, suggerendo che i livelli di guasto corrispondenti sono meno rappresentati o meno probabili.
- Le colonne $prob_0$, $prob_1$ e $prob_3$ mostrano una distribuzione intermedia, indicando che i dati sani e quelli relativi a guasti leggeri associati a queste probabilità sono presenti, ma non predominanti.

- La colonna $prob_4$ ha un numero molto basso di occorrenze, suggerendo che il livello di guasto associato a questa probabilità è raro o non significativo nei dati.

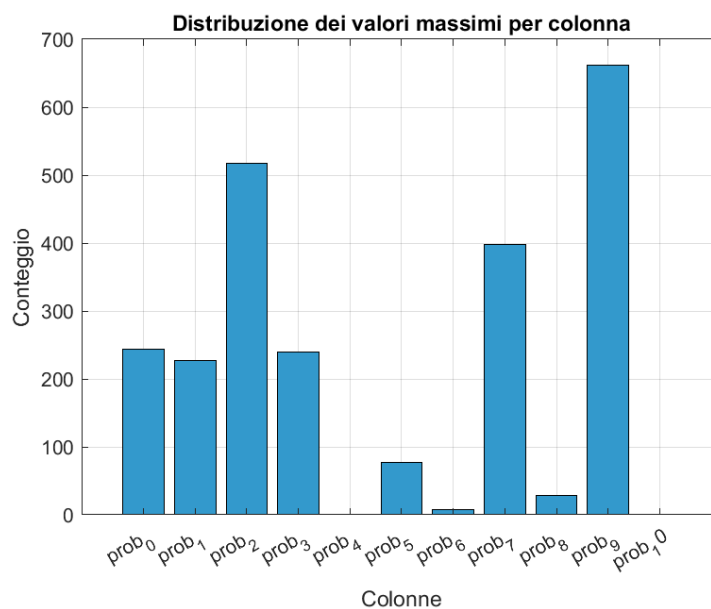


Figura 4.5: Occorrenze file submission_cnn_svm.m

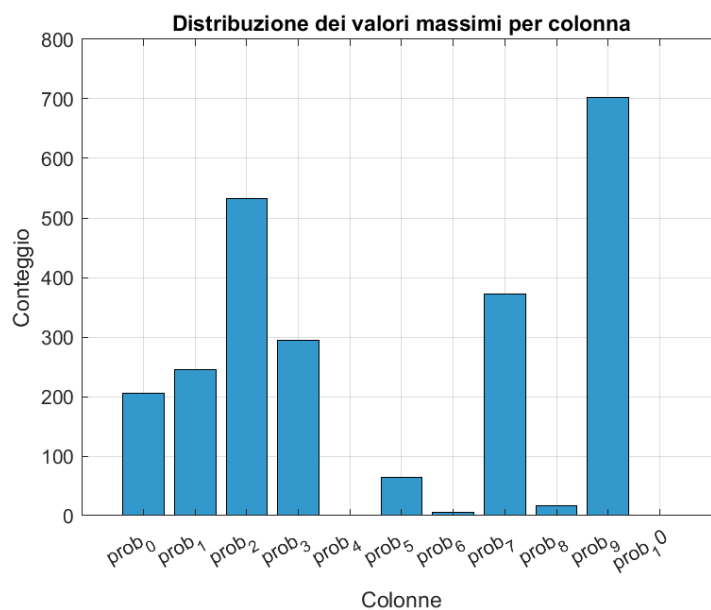


Figura 4.6: Occorrenze file submission_validation_cnn_svm.m

Analizzando più nel dettaglio, la colonna $prob_5$, non essendo presente come livello di guasto nei dati di training, segue l'euristica del modello espressa in precedenza. È quindi probabile che contenga occorrenze identificate come guasti legati alla colonna $prob_4$, ma ritenute di gravità maggiore.

Inoltre, la colonna $prob_{10}$ è rimasta sempre vuota a causa della mancanza della colonna $prob_9$ all'interno del training.

Riassumendo, i guasti con il maggior numero di occorrenze si collocano agli estremi, risultando principalmente di basso o alto livello.

4.4 Conclusioni

I risultati ottenuti dal modello di classificazione mostrano una solida capacità di distinzione tra le diverse classi di guasto, con un'accuratezza di validazione del 84,78%, che riflette una buona generalizzazione ai dati non visti durante l'addestramento. L'andamento dell'accuratezza e della funzione di costo durante il processo di training conferma che l'ottimizzazione ha proceduto in modo efficace, con una progressiva riduzione dell'errore e un miglioramento delle performance di classificazione.

Infine, l'analisi della matrice di confusione evidenzia che il modello ha effettuato un buon lavoro di classificazione.